GT6B: Final Project Documentation

Members:

Agravante, Maria Karylle V.

Clerigo, Rome Theodore G.

De Chavez, Ma. Angelica Pauleen R.

Paña, Jasmine Ysobel P.

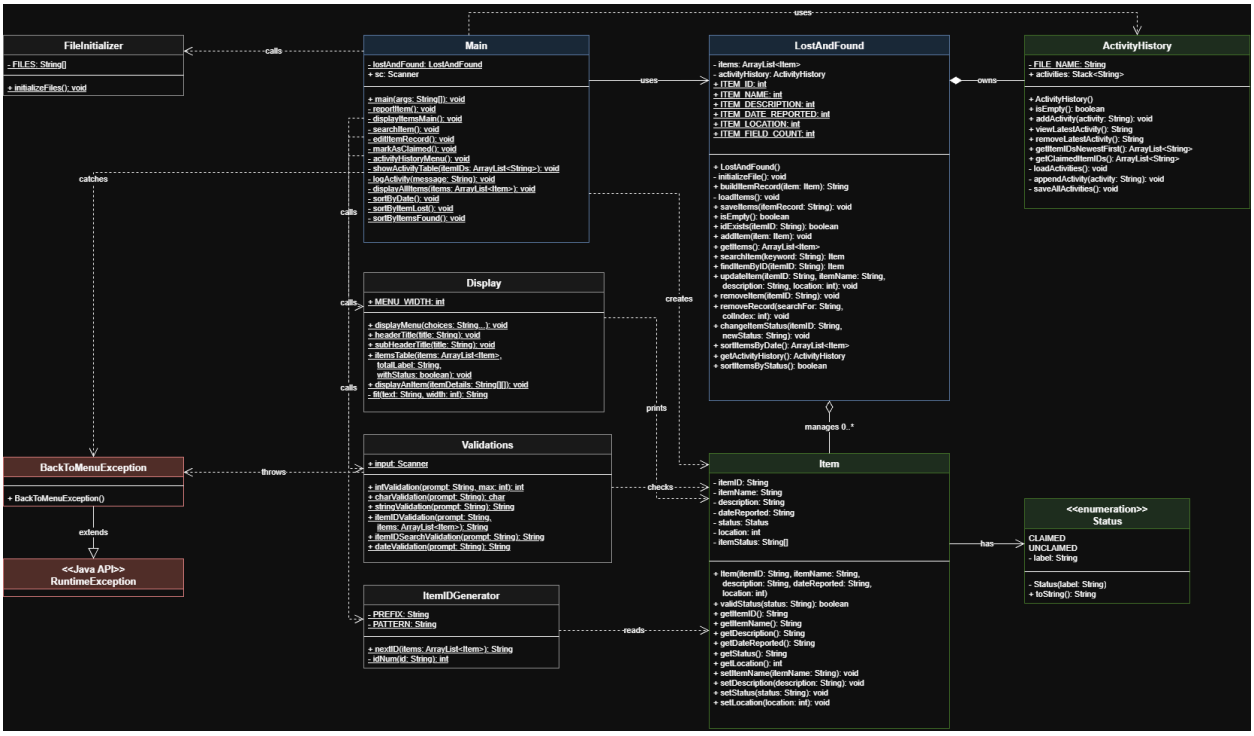
Peñaredondo, Clark Adrian S.

Introduction

The Campus Lost and Found Management System is a program that manages students' lost items on campus. This program will assist the admin in handling records of the reported items through item logs, as well as monitoring the recording and claiming of items. This will also be of help for students who are having a hard time finding their lost belongings, providing them a place to look for their items.

Through the program features such as recording of lost items, searching and sorting of items, as well as updating and displaying reported unclaimed items, the users are expected to have an easier and faster way to manage all the items and improve the overall efficiency of the lost and found process.

Program Design



[Link of UML Diagram]

The UML class diagram shows the structure of the Campus Lost and Found Management System, a Java console application used to report, display, search, edit, and claim lost items. It contains ten classes, grouped by responsibility, plus Java's built-in RuntimeException, which one of them extends. Every field and method is shown with its visibility, name, parameter types, and return type, such as addItem(item: Item): void. Signatures that are too long for a box are wrapped onto indented lines.

Main is the entry point and controls the menu. It holds a LostAndFound object as a private static field and a public Scanner field named sc. It has one method for each menu option, such as reportItem, searchItem, editItemRecord, and markAsClaimed. It also has methods for displaying and sorting items (displayAllItems(items: ArrayList<Item>), sortByDate, sortByItemLost, sortByItemsFound), and logActivity(message: String) for recording actions.

Main creates new Item objects when an item is reported, and it uses ActivityHistory directly when showing the activity menu.

LostAndFound handles the data. It keeps an ArrayList<Item>, reads and writes ITEMS.txt, and provides adding, searching, updating, removing, and sorting, such as searchItem(keyword: String): Item and removeRecord(searchFor: String, colIndex: int). Its updateItem method takes the item ID, new name, new description, and new location. Its constants (ITEM_ID, ITEM_NAME, ITEM_DESCRIPTION, ITEM_DATE_REPORTED, ITEM_LOCATION, and ITEM_FIELD_COUNT) tell it which position each field has in a saved record. It also owns an ActivityHistory object, which keeps a Stack<String> of logged actions and saves them to ACTIVITY.txt.

Each Item stores one report: ID, name, description, date reported, location, and a Status. Its constructor takes five parameters (ID, name, description, date reported, and location), and it sets the status to unclaimed by default. It also has a private itemStatus array holding the valid status words. Status is an enumeration whose value is either CLAIMED or UNCLAIMED, and it has a private constructor, Status(label: String), that stores a display label.

Four helper classes support the system, and all of their members are static. Display formats menus, headers, and tables. Its displayMenu(choices: String...) method accepts any number of menu options, and its itemsTable method takes the items, a total label, and a withStatus flag that decides whether the status column is shown. Validations checks user input (numbers, text, y/n answers, item IDs, and dates) and throws BackToMenuException when the user enters 0. ItemIDGenerator creates sequential IDs in the format ITM-0001 from the current list of items. FileInitializer creates the data files if they do not yet exist. BackToMenuException extends RuntimeException, and Main catches it to return to the main menu.

In the diagram, + marks a public member, - marks a private member, and underlined members are static. A solid arrow means a class holds another class as a field, such as Main holding LostAndFound and Item holding Status. A dashed arrow means a class only uses another inside its methods, such as Main calling Display, Validations, ItemIDGenerator, and

FileInitializer, or Display printing Item objects. A filled diamond shows composition, where LostAndFound owns ActivityHistory. An empty diamond shows aggregation, where LostAndFound manages many Item objects (0..*). A hollow triangle shows inheritance. All lines are drawn straight with sharp corners.

Code Inventory

1. Control Structures

Control Structure	Where It Is Used	Aspect of the Problem
while loop	Main.main()	Keeps the main menu running until the user chooses Exit (isExit becomes true).
while loop	Validations (intValidation, charValidation, stringValidation, itemIDValidation, itemIDSearchValidation, dateValidation)	Repeats the prompt until the user enters valid input.
while loop	LostAndFound.loadItems(), ActivityHistory.loadActivities()	Reads the text files line by line until the end of file.
while loop (inside insertion sort)	LostAndFound.sortItemsByDate(), sortItemsByStatus()	Shifts larger elements to the right while looking for the correct position of the current item.
for loop	LostAndFound.sortItemsByDate(), sortItemsByStatus(); Display.displayMenu(); ActivityHistory.getRecentActivities(), Display.containerSelection()	Traverses lists by index: the outer pass of the insertion sort, numbering menu options, and printing activities from newest to oldest.
Enhanced for (for-each) loop	Main (activityHistoryMenu, claimedFirst, sortByItemLost, sortByItemsFound); LostAndFound (idExists, findItemByID, getContainerItemCount, getItems); Display.itemsTable(itemsTable, containerSelection, itemsContainer, displayAnItem); ItemIDGenerator.nextID(), ActivityHistory.getClaimedItemIDs(), Validations.itemIDValidation(),	Goes through every Item or array element for filtering, searching, displaying, or finding the highest ID.
if / else if / else	Across all classes	Decision making: empty-list checks, null checks for not-found items, status checks (Claimed or Unclaimed), and input validation rules.

switch (arrow-style)	Main.main(), Main.displayItemsMain()	Routes the user's menu choice to the matching method.
switch (traditional with break)	Main.editItemRecord()	Chooses which field to edit: name, description, or container.
Ternary operator ? :	Main.markAsClaimed()	Converts the y/n answer into the status "Claimed" or "Unclaimed".
continue	Validations methods; LostAndFound.loadItems(); LostAndFound.getItems()	Skips an invalid input, blank line, or removed record and goes to the next iteration.
break	Main.editItemRecord() (switch cases); Validations.itemIDValidation()	Leaves a switch or loop once the job is done.
Early return	Main methods (empty list or item not found); LostAndFound.removeItem(), LostAndFound.loadItems(), ActivityHistory.loadActivities() (return if the file doesn't exist), ActivityHistory.addActivity() (return on empty activity), and viewLatestActivity() (returns null if empty).updateItem()	Stops a method immediately when there is nothing to process.
try / catch	Validations.intValidation() (NumberFormatException); Validations.dateValidation() (DateTimeParseException); LostAndFound.loadItems(); FileInitializer; LostAndFound.saveItems(), rewriteFile(), ActivityHistory.loadActivities(), and appendActivity()	Handles bad number or date input and file errors without crashing the program.
try-with-resources	LostAndFound (loadItems, saveItems, rewriteFile); ActivityHistory (loadActivities, appendActivity)	Opens BufferedReader, BufferedWriter, and FileWriter objects and closes them automatically.
throw (custom exception)	Validations methods	Throws BackToMenuException when the user types 0 to return to the main menu.

Ternary operator	Main.markAsClaimed(); Display.itemsTable()	Converts the y/n answer into the status "Claimed" or "Unclaimed", and chooses whether the item table includes the Status column.
Throw (built-in exception)	FileInitializer.initializeFiles()	Throws IllegalStateException when the data folder or data files cannot be created, stopping the program since it cannot run without them.

2. Libraries (Imports)

Library / Class	Package	Used In	Description
Scanner	java.util	Main, Validations	Reads keyboard input.
ArrayList	java.util	Main, LostAndFound, Validations, Display, ItemIDGenerator, ActivityHistory	Resizable list that stores all Item objects and other collections of records.
Stack	java.util	ActivityHistory	Last-in-first-out structure that stores the activity log so the latest activity is on top.
LocalDate	java.time	Main, Validations	Gets today's date for the date reported and validates entered dates.
LocalDateTime	java.time	Main	Gets the current date and time for the activity log timestamp.
DateTimeFormatter	java.time.format	Main, Validations	Formats and parses dates and times using patterns such as yyyy-MM-dd HH:mm.
ResolverStyle	java.time.format	Validations	Applies STRICT date checking so impossible dates (such as February 30) are rejected.

DateTimeParseException	java.time.format	Validations	Exception caught when a date cannot be parsed.
File	java.io	LostAndFound, FileInitializer, ActivityHistory	Represents ITEMS.txt and ACTIVITY.txt; used to check, create, and open the files.
FileReader / BufferedReader	java.io	LostAndFound, ActivityHistory	Reads the data files efficiently, one line at a time.
FileWriter / BufferedWriter	java.io	LostAndFound, ActivityHistory	Writes or appends records to the data files.
IOException	java.io	LostAndFound, FileInitializer, ActivityHistory	Exception caught for file input/output problems.

3. Variables

Variable	Type	Class / Method	Description
lostAndFound	LostAndFound (static)	Main	Main object that manages all items and the activity history.
sc	Scanner	Main	Scanner declared in Main.
input	Scanner (static)	Validations	Shared scanner used by all validation methods.
isExit	boolean	Main.main()	Flag that stops the main menu loop.
menu, choice	int	Main	Store the user's menu selection.
items	ArrayList<Item>	Main, LostAndFound	Holds all item records in memory.
name, description, date, container, itemID	String / int	Main.reportItem()	Hold the details of the item being reported.

oldName, oldDescription, oldContainer	String / int	Main.editItemRecord()	Keep the previous value so the change can be shown and logged.
status	String	Main.markAsClaimed()	Holds "Claimed" or "Unclaimed" from the user's y/n answer before it is applied to the item.
status	ItemStatus (private)	Item	Stores the item's current state (CLAIMED or UNCLAIMED); starts as UNCLAIMED.
claimedIDs	ArrayList<String>	Main, ActivityHistory	List of item IDs that were claimed according to the activity log.
time	String	Main.logActivity()	Formatted timestamp placed in each activity entry.
itemID, itemName, description, dateReported, status, container	String / int (private)	Item	Attributes of an item.
ITEM_ID, ITEM_NAME, ITEM_DESCRIPTION, ITEM_DATE_REPORTED, ITEM_CONTAINER, ITEM_FIELD_COUNT	static final int	LostAndFound	Constants for the column positions in a record and the number of fields.
items, activityHistory	ArrayList<Item>, ActivityHistory (private)	LostAndFound	Data kept by the manager class.

itemRecord	String[]	LostAndFound.buildItemRecord()	Array of fields joined with "|" to form one line in the file.
data	String[]	LostAndFound.loadItems()	Fields obtained by splitting a line from the file.
activities	Stack<String>	ActivityHistory	Stores activity log entries.
FILE	static final File (private)	ActivityHistory	Refers to the activity log file (ACTIVITY.txt), taken from FileInitializer.ACTIVITY_FILE.
FILES	static final String[] (private)	FileInitializer	Array of the data files (ITEMS.txt and ACTIVITY.txt) that must exist at startup.
ITEMS_FILE, ACTIVITY_FILE	static final File	FileInitializer	The item records file (data/ITEMS.txt) and the activity log file (data/ACTIVITY.txt).
PREFIX, PATTERN	static final String	ItemIDGenerator	The "ITM-" prefix and the regular expression for a valid ID.
max	int	ItemIDGenerator.nextID()	Highest ID number found so far.
MENU_WIDTH	static final int	Display	Width (121) of the lines and headers in the output.

isValid... flags (isValidMenu, isValidChar, isValidString, isValidID, isValidDate)	boolean	Validations	Control the loops that repeat until input is valid.
containerStatus	ContainerStatus (private)	Item	Stores whether the item's container is Full, Near Full, or Available; starts as Available.
CONTAINER_COUNT, CONTAINER_CAPACITY	static final int	LostAndFound	Number of containers (5) and the maximum number of items each container can hold (50).
ITEMS_FILE, ACTIVITY_FILE	static final File	FileInitializer	The item records file (data/ITEMS.txt) and the activity log file (data/ACTIVITY.txt).
label	String (private final)	ContainerStatus, ItemStatus	Readable display text for each enum constant (e.g. "Near Full", "Claimed").
count	int	LostAndFound.getContainerItemCount()	Running count of unclaimed items found in the given container.
line	String	LostAndFound.loadItems(), ActivityHistory.loadActivities()	The current line read from the file.
ctr, remaining	int	Display.containerSelection()	Number of items in a container and how many slots are left.
containerStatus	String	Display.containerSelection()	Status label printed beside each container.

labelWidth	int	Display.displayAnItem()	Length of the longest label, used to align the item details.
recentActivities	ArrayList<String>	ActivityHistory.getRecentActivities( )	Holds up to 10 of the latest activities, newest first.
startIndex	int	ActivityHistory.getRecentActivities( )	Index where the loop stops so only the last 10 activities are taken.
txt	String	Validations methods	The raw text entered by the user before it is checked.
parsedDate	LocalDate	Validations.dateValidation()	The entered date after parsing, used to check that it is today.
dataDirectory	File	FileInitializer.initializeFiles()	The data folder where ITEMS.txt and ACTIVITY.txt are stored.

4. Functions

4.1 Main

Method	Description
main(String[] args)	Entry point; initializes the files, creates the LostAndFound object, and runs the main menu loop.
reportItem()	Asks for item details, generates an ID, saves the new item, and logs the activity.
displayItemsMain()	Shows the display submenu (all items, sort by date, lost items, found items).
searchItem()	Searches for an item by ID and displays its details.
editItemRecord()	Edits an unclaimed item's name, description, or container and logs the change.

markAsClaimed()	Changes an item's status to Claimed after confirmation and logs it.
activityHistoryMenu()	Updates statuses from the activity log and displays items with claimed ones first.
claimedFirst(items)	Returns a new list with claimed items placed before unclaimed ones.
logActivity(message)	Adds a timestamped entry to the activity history.
displayAllItems(items)	Displays all items in a table.
sortByDate()	Displays items sorted from oldest to newest.
sortByItemLost()	Displays only unclaimed items.
sortByItemsFound()	Displays only claimed items.

4.2 Validations

Method	Description
intValidation(prompt, max)	Accepts a whole number from 1 to max; rejects letters, symbols, and leading zeros; 0 returns to the menu.
charValidation(prompt)	Accepts only y or n.
stringValidation(prompt)	Accepts non-empty text and cleans extra spaces.
itemIDValidation(prompt, items)	Accepts an ID in the format ITM-0001 that does not already exist.
itemIDSearchValidation(prompt)	Accepts an ID in the format ITM-0001 for searching.
dateValidation(prompt)	Accepts a valid date in YYYY-MM-DD format that is today's date.

4.3 LostAndFound

Method	Description
LostAndFound() (constructor)	Creates the file if needed and loads the items.
initializeFile()	Creates ITEMS.txt if it does not exist.
buildItemRecord(item)	Converts an Item into a "|"-separated line.
loadItems()	Reads ITEMS.txt and rebuilds the list of Item objects.
saveItems(itemRecord)	Appends a record to ITEMS.txt.

isEmpty()	Tells whether there are no items.
isContainerFull(container)	Checks if number of items in a container is at max or more than the capacity.
idExists(itemID)	Checks if an ID is already used.
addItem(item)	Adds an item to the list and saves it to the file.
getItems()	Returns the list of items.
findItemByID(itemID)	Linear search by exact ID.
updateItem(itemID, itemName, description, container)	Updates an item's name, description, and container, then rewrites the whole file.
removeItem(itemID)	Removes an item from the list and the file.
rewriteFile()	Saves the entire items list back to the file, replacing whatever was there before.
getContainerItemCount(container)	Counts the unclaimed items currently stored in a given container.

4.4 Item

Method	Description
Item(...) (constructor)	Creates an item; its status starts as "Unclaimed".
getItemID(), getItemName(), getDescription(), getDateReported(), getStatus(), getContainer()	Getters that return the attributes.
setItemName(), setDescription(), setStatus(), setContainer()	Setters that change the editable attributes.
getContainerStatus()	Returns the container's status label (Full, Near Full, or Available).
setContainerStatus(remaining)	Sets the container status based on remaining space: Full when none is left, Near Full when 10 or fewer slots remain, Available otherwise.

4.5 ActivityHistory

Method	Description

ActivityHistory() (constructor)	Loads saved activities from ACTIVITY.txt.
isEmpty()	Tells whether there are no activities.
addActivity(activity)	Pushes an activity to the stack and appends it to the file.
viewLatestActivity()	Returns the most recent activity without removing it (peek).
getRecentActivities()	Returns list of recent activities done in the program.
getClaimedItemIDs()	Finds the IDs of items marked as claimed in the log.
loadActivities()	Reads ACTIVITY.txt into the stack.
appendActivity(activity)	Adds one line to ACTIVITY.txt.

4.6 Display, ItemIDGenerator, FileInitializer, BackToMenuException

Class	Method	Description
Display	**displayMenu(choices...)**	Prints a numbered menu from a variable number of choices.
Display	**centerText(text)**	Prints text in uppercase, centered within menu width.
Display	**headerTitle(title)**	Prints a centered title between heavy lines.
Display	**subHeaderTitle(title)**	Prints a centered subtitle between dashes.
Display	**containerSelection(items)**	Shows each container's capacity and status so the user can choose where to store an item.
Display	**itemsTable(items, totalLabel, withStatus)**	Prints items in an aligned table with a total count.
Display	**itemsContainer(items, totalLabel)**	Prints items in a table without the container and status columns, with a total count.
Display	**displayAnItem(details, itemDetails)**	Prints the details of a single item as label : value.
Display	**fit(text, width)**	Shortens long text with "..." so the table stays aligned.
ItemIDGenerator	**nextID(items)**	Generates the next ID (for example ITM-0004).

ItemIDGenerator	**idNum(id)**	Extracts the number from an ID.
FileInitializer	**initializeFiles()**	Creates ITEMS.txt and ACTIVITY.txt if missing.
BackToMenuException	**BackToMenuException() (constructor)**	Creates the exception with the message "Back to main menu".

4.7 ContainerStatus and ItemStatus

Class	Method	Description
ContainerStatus	**ContainerStatus(label) (constructor)**	Stores the display label for each container status constant.
ContainerStatus	**toString()**	Returns the readable label instead of the constant name.
ItemStatus	**ItemStatus(label) (constructor)**	Stores the display label for each item status constant.
ItemStatus	**toString()**	Returns the readable label instead of the constant name.

5. Classes

Class	Description
Main	Contains the main method and the menu-driven user interface; connects the user's choices to the other classes.
Item	Model class that represents one lost or found item.
LostAndFound	Manager class that holds the list of items and handles searching, sorting, adding, updating, removing, and file saving or loading.
ActivityHistory	Keeps a log of actions (reported, edited, claimed) using a stack and a file.
Validations	Utility class that checks and cleans all user input.
Display	Utility class for formatting menus, headers, and tables.
ItemIDGenerator	Utility class that creates unique, sequential item IDs.

FileInitializer	Utility class that makes sure the data files exist at startup.
BackToMenuException	Custom exception class that signals the user wants to return to the main menu.
ContainerStatus	Enum that defines the fixed status values (Full, Near Full, Available), each with a display label used when printing.
ItemStatus	Enum that defines the possible states of an item (Claimed, Unclaimed), each with a display label used when printing.

6. Objects

Object	Class	Where Created	Description
lostAndFound	LostAndFound	Main.main()	The single manager object for the whole program.
item	Item	Main.reportItem()	The new item created from user input.
item	ItemStatus	LostAndFound.loadItems()	An Item rebuilt from each line of ITEMS.txt when the program loads saved records.
items (list)	ArrayList<Item>	LostAndFound	Collection of all Item objects in memory.
activityHistory	ActivityHistory	LostAndFound	Object that logs activities; owned by LostAndFound.
activities	Stack<String>	ActivityHistory	Stack object that holds the log entries.
sc, input	Scanner	Main, Validations	Objects that read keyboard input.
file	File	LostAndFound, FileInitializer, ActivityHistory	Represents a data file.
CLAIMED, UNCLAIMED	ItemStatus	ItemStatus (created automatically)	The only two item status objects; created once when the enum is first used.

FULL, NEAR_FULL, AVAILABLE	ContainerStatus	ContainerStatus (created automatically)	The only three container status objects; created once when the enum is first used.
ITEMS_FILE, ACTIVITY_FILE	File	FileInitializer	File objects shared by LostAndFound and ActivityHistory to locate the data files.
dataDirectory	File	FileInitializer.initializeFiles()	Represents the data folder, created if it does not exist.
parsedDate	LocalDate	Validations.dateValidation()	The date object produced from the user's input.
recentActivities, claimedIDs	ArrayList<String>	ActivityHistory	Lists built from the activity log for display and claimed-status checking.
reader, out, writer	BufferedReader, BufferedWriter, FileWriter, Scanner	File-handling methods	Objects used to read from or write to files.
formatter	DateTimeFormatter	Validations.dateValidation()	Strict date pattern used for parsing.
BackToMenuException	RuntimeException subclass	Validations	Exception object thrown when the user enters 0.

7. Object-Oriented Programming Principles

Principle	Where Used	Discussion
Classes and Objects	Item, LostAndFound, ActivityHistory and the others	Each real-world concept (an item, the lost and found office, the activity log) is a class. Objects are created from them, for example each reported item becomes an Item object stored in the items list.

Encapsulation	Item, LostAndFound, ActivityHistory	Fields are private and accessed only through getters and setters. For example, an item's ID has no setter, so it cannot be changed after creation; activities in ActivityHistory can only be changed through addActivity().
Inheritance	BackToMenuException extends RuntimeException	The custom exception inherits the behavior of RuntimeException (it can be thrown without being declared) and passes its message to the parent through super(...).
Constructors	Item, LostAndFound, ActivityHistory, BackToMenuException	Set up objects in a valid starting state, such as a new Item starting as "Unclaimed" and LostAndFound loading saved data when created.
Enum	ItemStatus, ContainerStatus (used in Item and Display)	Restricts item and container status to a fixed set of valid values, so invalid states like “Lost” or “Half Full” cannot exist. Each constant carries a display label for printing.

Concepts of Data Structures and Algorithms

The Lost and Found Management System is a program designed to help schools record lost items and help students locate their belongings. For these purposes, the program utilized the following Data Structures and Algorithms concepts:

Concepts	Time Complexity	Space Complexity	Description & Explanation
Stack	O(1) for push and peek Explanation: ➔ Adding or viewing the top activity takes the same time regardless of size.	O(n) Explanation: ➔ It stores every activity.	A linear data structure that follows the Last In, First Out (LIFO) principle in which the last value inserted is the first one removed. In the program, it is used to track ongoing activities, particularly the most recent changes as well as system activity. They are then inserted at the top of the records, which can be viewed in the Activity History. It is implemented to make it easier for the users to see the latest tasks completed by the system, which is ideal for monitoring activity logs, specifically the item's handling record.
Linear Search	Best: O(1) Average: O(n) Worst: O(n) Explanation: ➔ It may check every item before finding a match.	O(1) Explanation: ➔ It uses no extra storage.	Linear search is a searching algorithm that sequentially looks at each value in the given collection until the desired item is found. In the program, it is used to search for the item reported through its unique ID, which is generated after it was surrendered to the lost and found. Thereafter, the program will display the matched item's relevant information such as its ID, name, description, container, and date reported.

			It is implemented because the program will only have a small amount of records to be handled so having it checked one by one would not be a problem.
Insertion Sort	Best: O(n) Average: O(n^2) Worst: O(n^2) Explanation: ➔ Each item may be compared with all the items before it.	O(1) Explanation: ➔ It sorts within the same list.	Insertion Sort is a sorting algorithm that moves the smallest element one at a time to a place where all sorted values are being stored. In the program, items are collected in the order they were reported, from oldest to newest. They are displayed after the sorting is done. It is implemented because it functions best when the records are already almost sorted, considering that items are stored based on the date it is reported.

Github Link: https://github.com/jasmineyso/datastrc-finals-3.git